

Java Servlets & Apache Tomcat — Assignment Answers

Topics: Tomcat setup, Dynamic Web Project, servlet lifecycle, web.xml configuration

Q1. Setting up Apache Tomcat + Dynamic Web Project in Eclipse

- 1 **Download & install Tomcat:** Get Apache Tomcat (e.g. version 10.x) from tomcat.apache.org, extract it to a folder like C:\apache-tomcat-10.1 (or /opt/tomcat on Linux/Mac). No separate 'installation' is needed for the zip version — it just needs to be unzipped.
- 2 **Register Tomcat in Eclipse:** Open Eclipse → Window → Preferences → Server → Runtime Environments → Add → select Apache Tomcat v10.x → Browse to the folder where you extracted Tomcat → Finish.
- 3 **Create a Dynamic Web Project:** File → New → Dynamic Web Project → give it a name (e.g. MyWebApp) → set the Target Runtime to the Tomcat you just added → keep 'Generate web.xml deployment descriptor' checked → Finish.
- 4 **Add a simple HTML page:** Right-click WebContent (or src/main/webapp) → New → HTML File → name it index.html → add simple content like `<h1>Hello from Tomcat!</h1>`.
- 5 **Run on server:** Right-click the project → Run As → Run on Server → select the configured Tomcat server → Finish. Eclipse starts Tomcat and opens the browser automatically.
- 6 **Verify:** Open a browser and visit <http://localhost:8080/MyWebApp/> (or <http://localhost:8080/> if deployed as the ROOT app) — the index.html content should be displayed.

Q2. WelcomeUserServlet — displays 'Welcome, User!'

File: `src/WelcomeUserServlet.java`

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/welcomeuser")
public class WelcomeUserServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<h2>Welcome, User!</h2>");
        out.println("</body></html>");
    }
}
```

Accessing **`http://localhost:8080/MyWebApp/welcomeuser`** in the browser triggers `doGet()` and displays 'Welcome, User!' on the page.

Q3. Overriding init(), service(), destroy() with console logging

The lifecycle methods are overridden to print a message to the console (server log) each time they run, so we can observe when Tomcat calls each one.

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletConfig;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;

@WebServlet("/welcomeuser")
public class WelcomeUserServlet extends HttpServlet {

    @Override
    public void init(ServletConfig config) throws ServletException {
        super.init(config);
        System.out.println("init() called - Servlet is being initialized");
    }

    @Override
    public void service(ServletRequest request, ServletResponse response)
        throws ServletException, IOException {
        System.out.println("service() called - Handling a client request");
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body><h2>Welcome, User!</h2></body></html>");
    }

    @Override
    public void destroy() {
        System.out.println("destroy() called - Servlet is being taken out of service");
    }
}
```

Observed behaviour: init() runs only once — the first time the servlet is requested (or at server startup, if load-on-startup is set). service() runs on every single request to /welcomeuser. destroy() runs only once — when Tomcat shuts down or the web application is undeployed/reloaded.

Q4. web.xml Mapping + init-param 'appName'

File: WebContent/WEB-INF/web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
    https://jakarta.ee/xml/ns/jakartaee/web-app_5_0.xsd"
  version="5.0">

  <servlet>
    <servlet-name>WelcomeUserServlet</servlet-name>
    <servlet-class>WelcomeUserServlet</servlet-class>
    <init-param>
      <param-name>appName</param-name>
      <param-value>MyWebApp</param-value>
    </init-param>
  </servlet>

  <servlet-mapping>
    <servlet-name>WelcomeUserServlet</servlet-name>
    <url-pattern>/welcomeuser</url-pattern>
  </servlet-mapping>

</web-app>
```

Note: since web.xml now declares the mapping, remove the `@WebServlet("/welcomeuser")` annotation from the Java class to avoid a duplicate mapping conflict.

Reading and displaying the init-param in the servlet:

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

public class WelcomeUserServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String appName = getInitParameter("appName");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<h2>Welcome, User!</h2>");
        out.println("<p>App Name: " + appName + "</p>");
        out.println("</body></html>");
    }
}
```

Visiting `/welcomeuser` now displays both 'Welcome, User!' and 'App Name: MyWebApp'.

Q5. web.xml Snippet — ReviewServlet for a Zomato-style App

Only the <servlet> and <servlet-mapping> entries are given below, as required (no Java code):

```
<servlet>
  <servlet-name>ReviewServlet</servlet-name>
  <servlet-class>ReviewServlet</servlet-class>
</servlet>

<servlet-mapping>
  <servlet-name>ReviewServlet</servlet-name>
  <url-pattern>/addreview</url-pattern>
</servlet-mapping>
```

This registers a servlet class named ReviewServlet under the logical name 'ReviewServlet', and maps any request to **/addreview** (e.g. when a user submits a food review form) to that servlet.